SOFTWARE ENGINEERING · MINI PROJECT

ReturnLoop

A returns and reverse-logistics centre for an e-commerce business: request, pickup, quality check, grading, refurbish / restock / scrap, refund.



PRESENTED BY 04

01 Chirag Seervi

02 Anshuman Singh

03 Aryan Singh

04 Aadi Srivastava

FLASK · SQLITE · THREE.JS

AGENDA

What we'll cover

02

01 Project planning & management — SLIDE 03

02 Functional requirements — SLIDE 06

03 Non-functional requirements — SLIDE 07

04 Use case diagram — SLIDE 08

05 Class diagram — SLIDE 09

06 Data flow diagram, level 0 — SLIDE 10

07 Data flow diagram, level 1 — SLIDE 11

08 Activity diagram — SLIDE 12

09 Sequence diagram: grading — SLIDE 13

10 State diagram — SLIDE 14

11 Sequence diagram: handoff — SLIDE 15

12 Layered architecture — SLIDE 16

13 Architecture: from UI to data — SLIDE 17

Vision, scope and stakeholders

03

Product vision

Give an online store one place to run every return, from the customer's doorstep to a graded item and a correct refund, with every step on record.

Objectives

- Enforce return rules automatically
- Grade items A / B / C and route them
- Compute refunds exactly, once
- Live tracking for the customer

In scope

- 6 roles, 13-status workflow
- Pickup booking, 2 attempts
- QC checklist, fraud check
- Refurbish & re-inspection
- Refund + inventory log
- Admin dashboard & flags

Out of scope

- Real payments / payment gateway
- Passwords & real accounts
- Courier API integration

Stakeholders

Customer	returns items, tracks refund
Pickup agent	collects parcels
Warehouse	checks parcels in
QC inspector	grades items
Refurbisher	repairs grade B
Admin	monitors, flags

Team

Chirag Seervi, Anshuman Singh, Aryan Singh, Aadi Srivastava

04

Incremental delivery in seven increments

Each increment was built, tested with **pytest**, run, and committed to **Git** before the next one started. Definition of done: all tests green and the app runs.

INC A
Core
Models, config, services, unit tests

INC B
Data
Seed data through real services

INC C
Customer
Orders, return, pickup, tracking

INC D
Operations
Agent, warehouse, QC, refurbisher

INC E
Admin
Dashboard, charts, flags

INC F
Experience
Design system, 3D scenes, effects

INC G
Hardening
Edge cases, docs, end-to-end demo

Why incremental?

Business rules first, UI later: the riskiest logic (state machine, refunds) was tested before any screen existed.

Work breakdown

Data model → services → routes → templates → visual design → testing & docs.

Tools

Python, Flask, SQLite, pytest, Git, VS Code, Playwright for screenshot checks.

Risks, and how we handled them

05

RISK	IMPACT	MITIGATION IN THE PRODUCT
Invalid status jumps (e.g. refund before QC)	HIGH	Single state machine; every change validated and logged in `status_history`
Refund paid twice	HIGH	Unique `refunds.return_id` + service check + one DB transaction
Double-clicks on submit	MEDIUM	Buttons disable on submit; server rejects the repeat via the state machine
Malicious or huge uploads	MEDIUM	Type, 3 MB size and file-signature checks; random file names
Device without WebGL / slow GPU	LOW	Static fallbacks; particles halve if frame rate < 40 fps
Rules change later (window, fees)	LOW	All rule constants live in `config.py`

QUALITY: 109 AUTOMATED TESTS · STATE MACHINE, REFUNDS, ELIGIBILITY, FLAGS, API, PAGES, FULL FLOW

Functional requirements

06

FR01
Role switch
Choose any of 6 roles and a user without a password; each page rejects the wrong role (403).

FR02
View orders
Customer sees delivered items with eligibility, or the reason it can't be returned.

FR03
Eligibility
Return allowed within 7 days of delivery, for returnable products, with no other open return.

FR04
Request return
Reason, comment (≤ 300 chars), optional photo (PNG/JPG/WEBP, ≤ 3 MB).

FR05
Book pickup
Date from tomorrow to +5 days, slot Morning / Afternoon / Evening; reschedule.

FR06
Collect parcel
Agent marks picked up or absent; the 2nd absence cancels the return.

FR07
Receive
Warehouse checks the parcel in; it joins the QC queue.

FR08
Inspect & grade
QC checklist, suggested grade, A/B/C; a mismatched item is rejected as fraud.

FR09
Refurbish
Grade B goes to repair, then re-inspection: pass → restock (70%), fail → scrap.

FR10
Refund
final = max(0, base × grade% − pickup fee), damaged/wrong-item exception, created once.

FR11
Track
Live status (polled every 5 s), history timeline and refund breakdown.

FR12
Dashboard
Totals, reasons and grades charts, frequent-returner flags (> 3 in 30 days).

FR13
Hand off
After each step, a card names the next desk and opens its page in one click.

FR14
Repair ticket
QC's findings become the fix list; ticked fixes are saved as the notes; past repairs show QC's verdict.

FR15
Live feed
The admin sees the latest status changes, newest first, with who made each one.

Non-functional requirements

07

NFR01
Performance
60 fps on a mid laptop, 30+ on phones; pixel ratio capped; 3D pauses off-screen.

NFR02
Reliability
Refund + inventory log + status change in one transaction; no double refunds.

NFR03
Security
Role guard on every blueprint; customers only see their own returns; upload validation.

NFR04
Usability
Plain-language errors and empty states; the handoff card always names the next step.

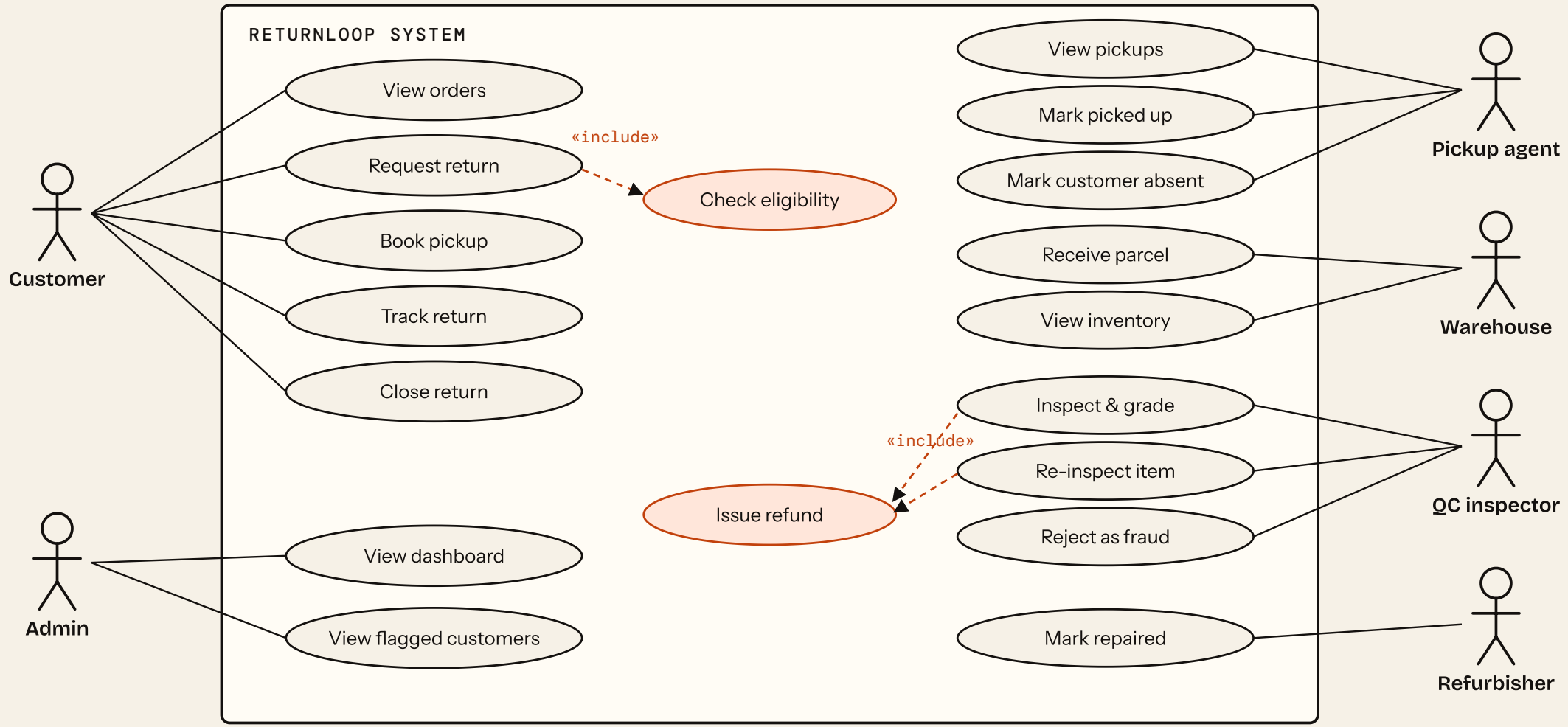
NFR05
Accessibility
WCAG AA contrast, keyboard-only use, visible focus, reduced-motion mode, text on every badge.

NFR06
Maintainability
Logic in services/, rules in config.py, thin routes; a pytest suite over services, pages and the full flow.

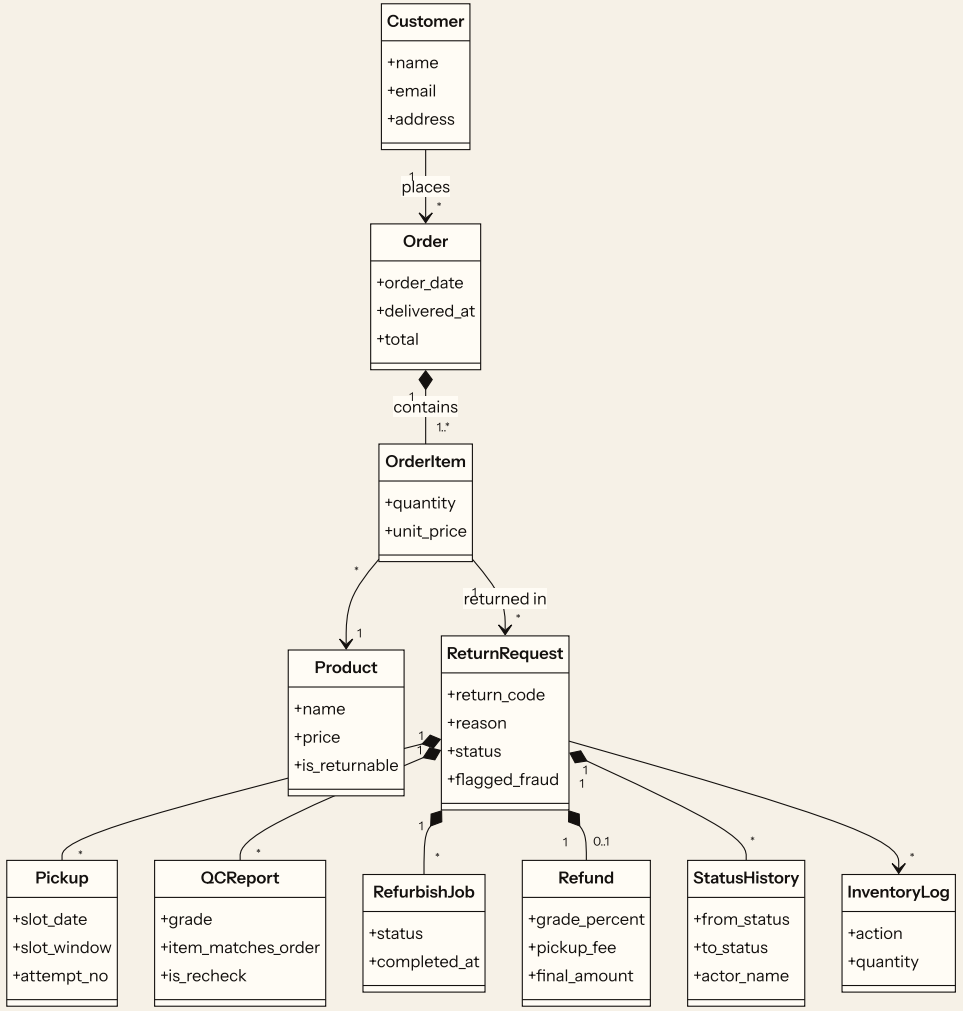
NFR07
Portability
Python 3.10+, SQLite, no build step: install, seed, run in 3 commands.

NFR08
Robustness
Static fallbacks without WebGL or internet; friendly 403 / 404 / 413 pages.

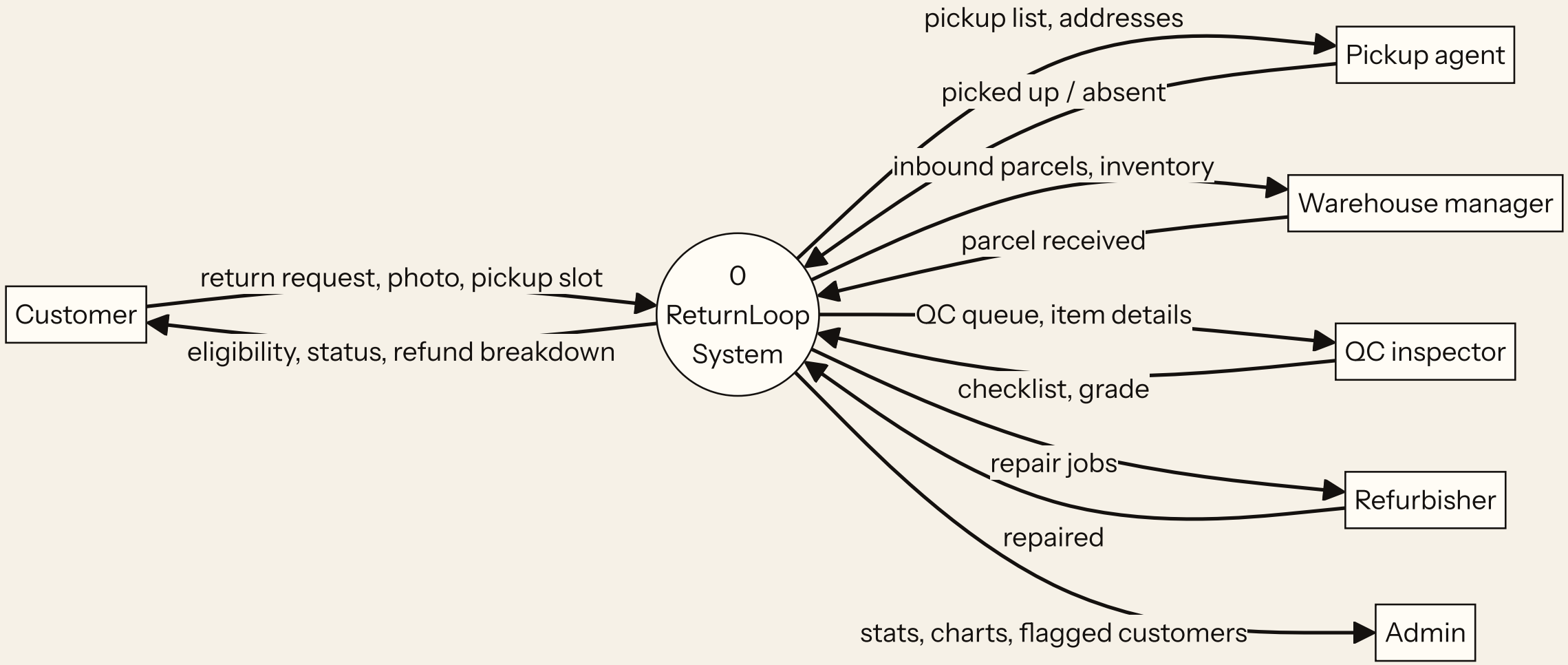
Who does what



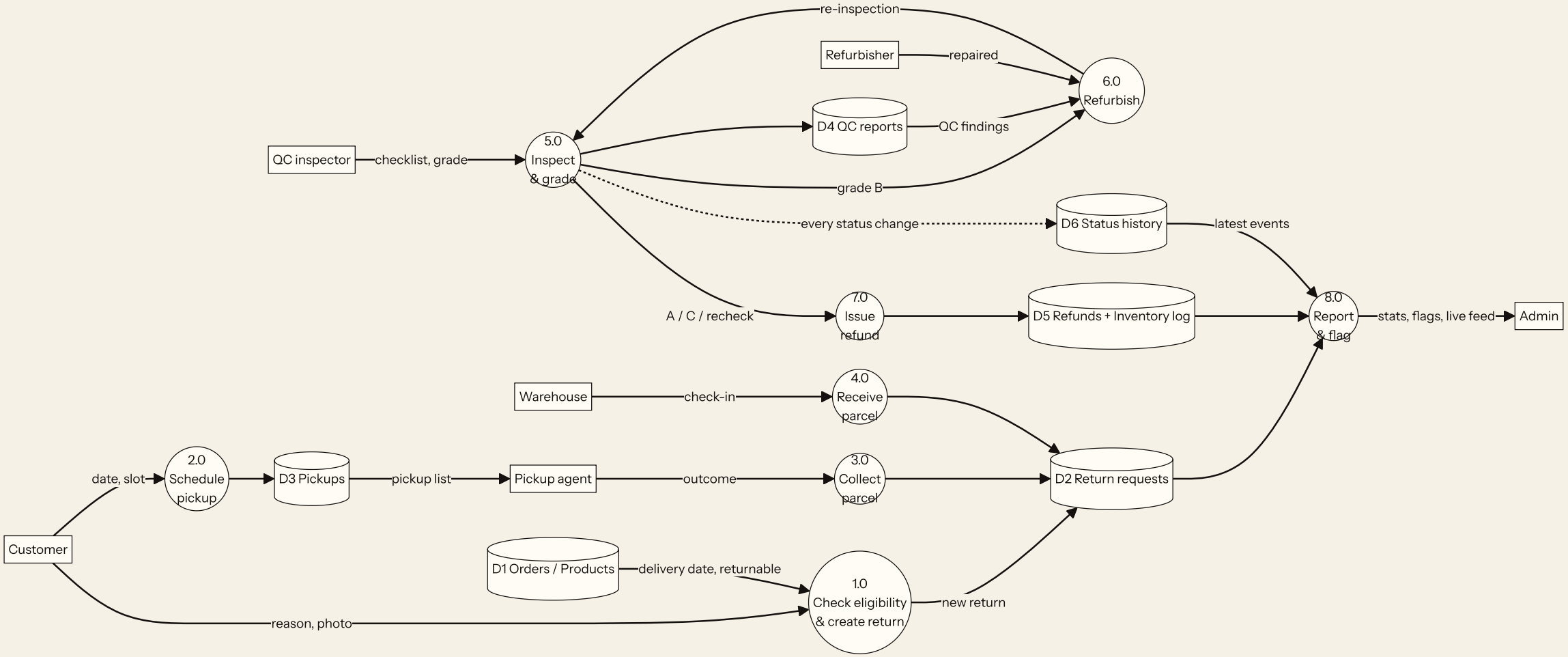
The domain model (models.py)



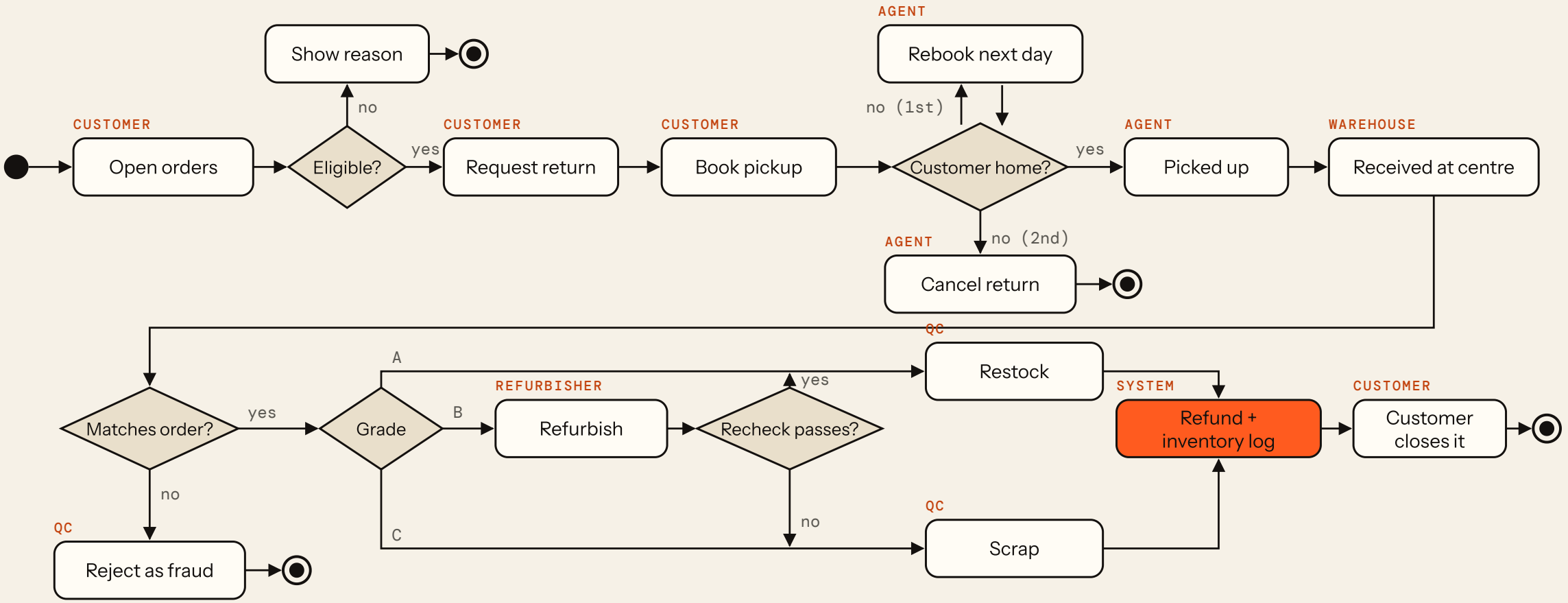
The system and the people around it



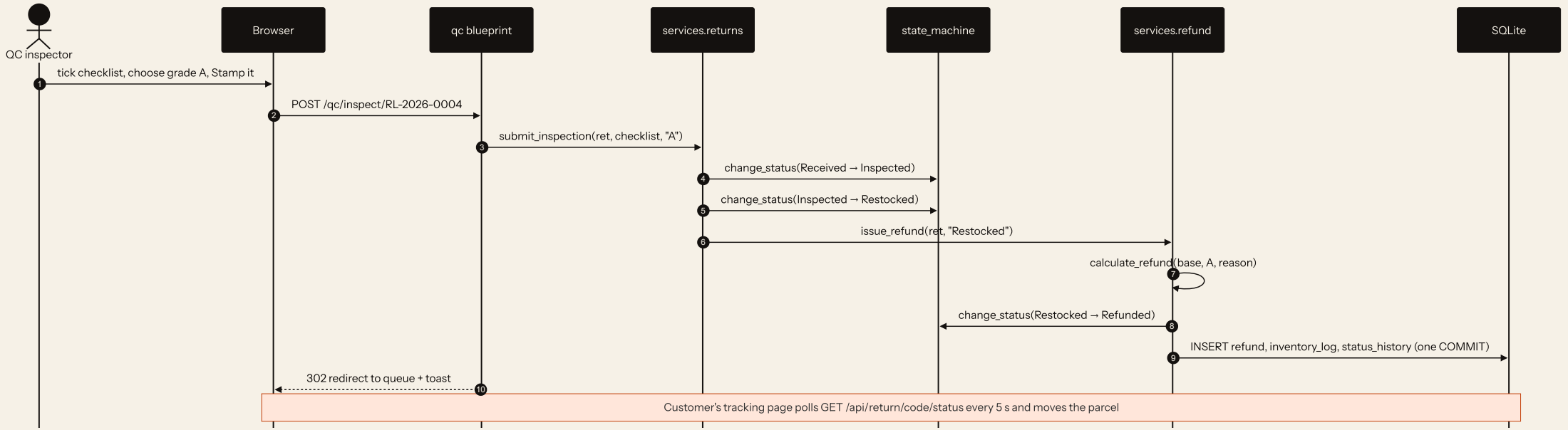
Processes and data stores



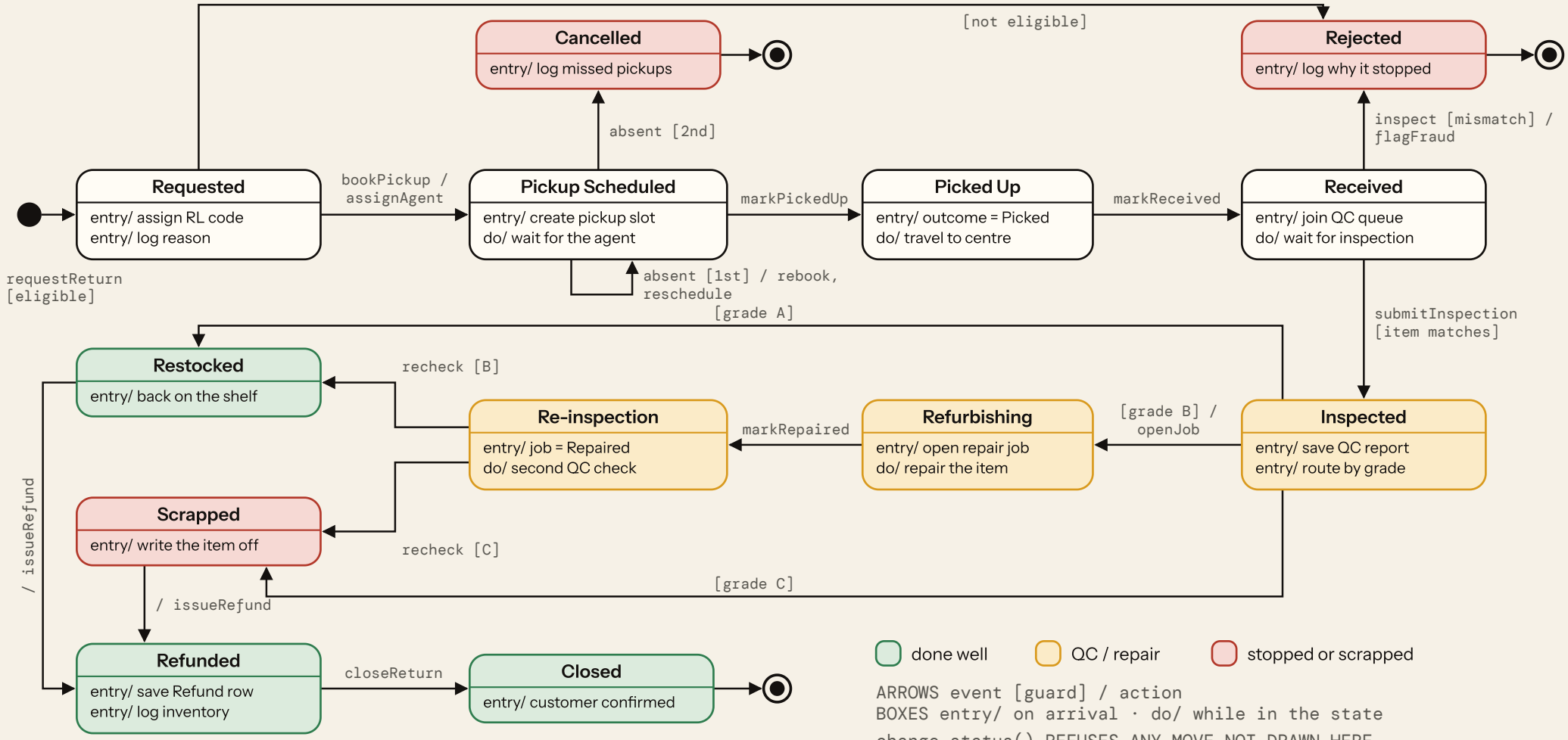
One return, start to finish



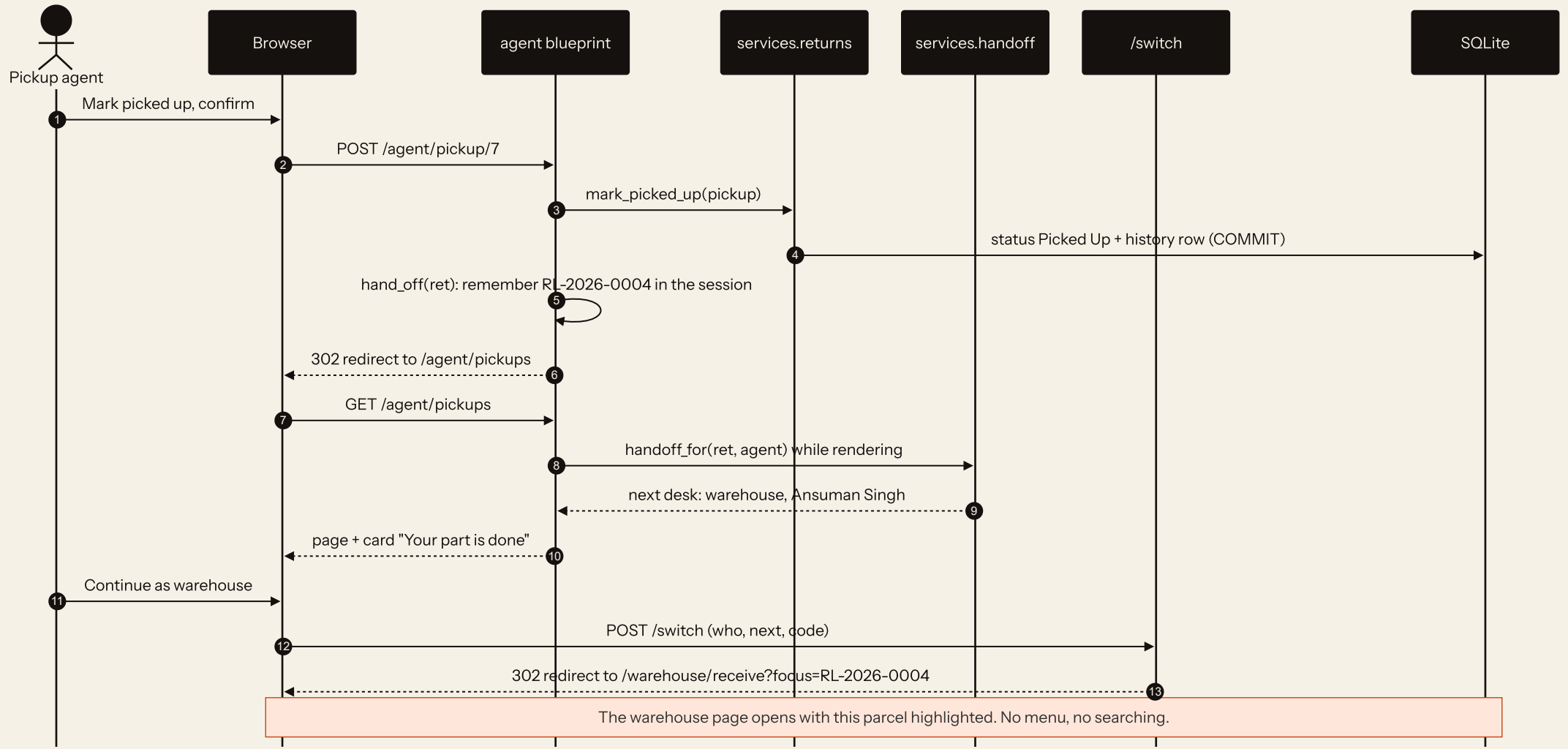
QC grades a return and the refund is issued



Every status a return can be in



Handing a return to the next desk



Layered architecture

L1	**Presentation**	Jinja2 templates · main.css · ES modules · Three.js scenes · GSAP · Chart.js	`templates/, static/`
L2	**Controller / routing**	Flask blueprints, one per role + JSON API; role guard on each	`blueprints/`
L3	**Business logic**	Workflow, state machine, eligibility, refund, flags, uploads, stats	`services/`
L4	**Configuration**	Every business rule as a named constant (window, fees, %, limits)	`config.py`
L5	**Data access**	Flask-SQLAlchemy ORM: 11 model classes and their relationships	`models.py`
L6	**Persistence**	SQLite database file, indexes on status, customer, history	`returnloop.db`

RULE: EACH LAYER ONLY TALKS TO THE ONE BELOW · ROUTES NEVER CONTAIN BUSINESS RULES

One click, layer by layer: “Request return”

1 UI
Browser
Form with reason tiles, comment and photo is submitted (button disables itself).

→

2 HTTP
POST /customer/return/new
Multipart request reaches Flask; session holds role + user id.

→

3 Guard
customer blueprint
before_request checks the role is customer, else friendly 403.

4 Validate
services.uploads
Photo type, size ≤ 3 MB and file signature are checked.

→

5 Rules
services.eligibility
7-day window, returnable product, no open return.

→

6 Logic
services.returns
Generates RL-2026-0004, writes the first status_history row.

7 ORM
models.ReturnRequest
SQLAlchemy turns objects into INSERT statements.

→

8 Data
SQLite
Rows committed in one transaction.

→

9 Back to UI
Redirect + Jinja2
Pickup page renders; tracking page later polls the JSON API.

END OF DECK · QUESTIONS?

Thank you.



Chirag Seervi · Anshuman Singh · Aryan Singh · Aadi Srivastava

```
pip install -r requirements.txt && python seed.py && python app.py
```